ALGORITHMS II (CSC711) · MINI PROJECT REPORT

Smart Delivery Network on Kalyani

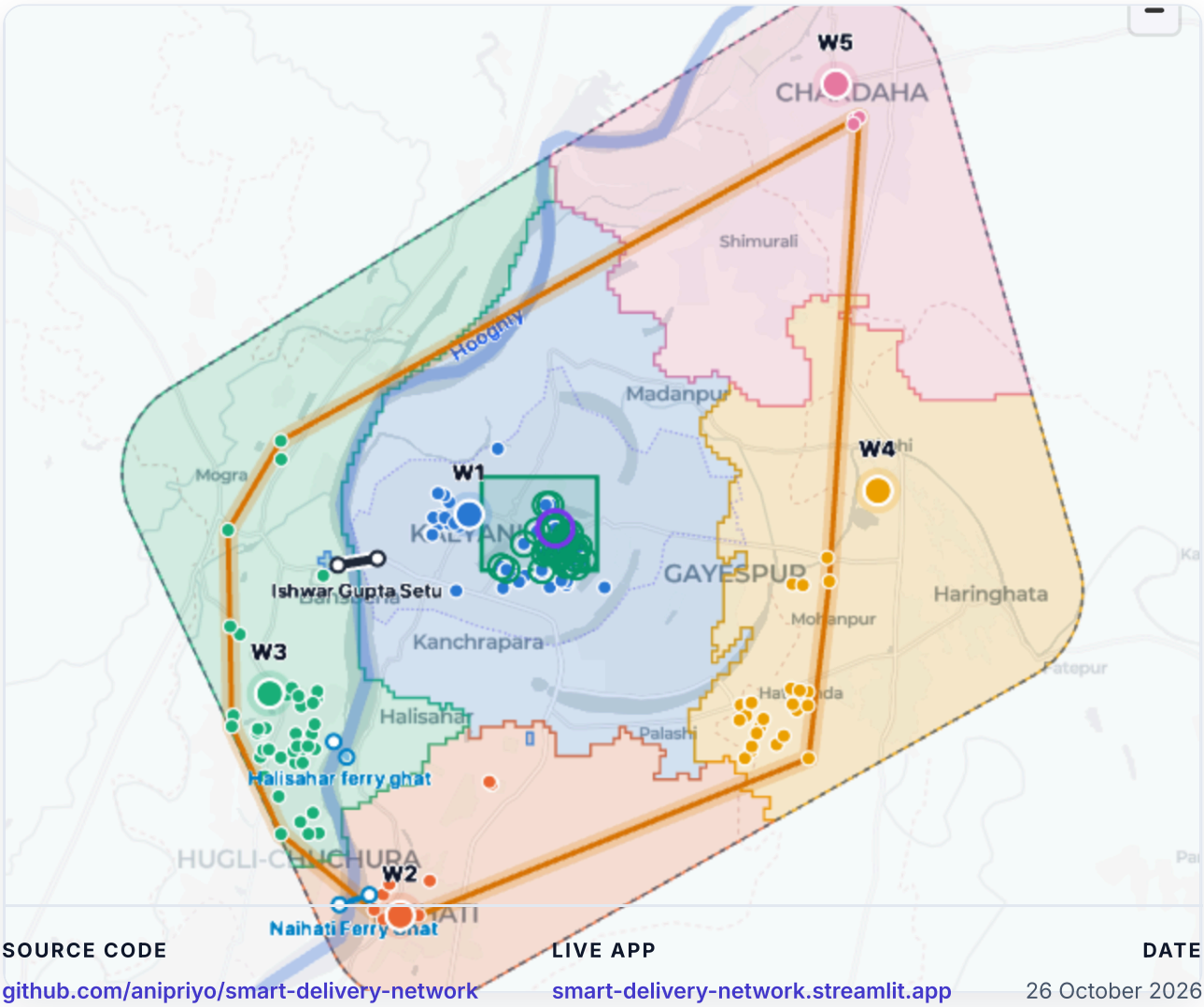
Computational geometry, shortest paths and route planning for deliveries on the real OpenStreetMap road network, on both banks of the Hooghly

Voronoi · KD-tree · Closest pair · Convex hull · Set cover · Dijkstra + routing · River check

Akash Chakraborty
Reg. 1059 · Roll CSE/23011

Anipriyo Mandal
Reg. 1065 · Roll CSE/23017

Arpan Mandal
Reg. 1075 · Roll CSE/23027



SOURCE CODE
github.com/anipriyo/smart-delivery-network

LIVE APP
smart-delivery-network.streamlit.app

DATE
26 October 2026

Contents

1 Problem Statement

2 Input Model

3 Algorithms Used

4 How the Algorithms Combine

5 Dashboard

6 Worked Example (default network)

7 Crossing the Hooghly, a Real-World Issue

8 Route Planner Example

9 Testing

10 Discussion

11 Data Sources and Attribution

12 Conclusion

A Running the Project (appendix)

Summary. A delivery network around Kalyani, West Bengal, built from 9,422 real OpenStreetMap places and 3,291 km of roads in a 401 km² area on both banks of the Hooghly. Five Algorithms II techniques plan it, namely Voronoi zones by road distance (who serves whom), a KD-tree (what is where), divide-and-conquer closest pair (what is near), the convex hull (how far it reaches) and greedy set cover by road (which sites to keep). A route planner finds the fastest round for a product to several places with Dijkstra and an exact Held-Karp order, or nearest neighbour with 2-opt for longer rounds. Everything is shown in an interactive Streamlit dashboard with step-by-step walkthroughs and pseudocode. With one road bridge along 27.5 km of river, a straight-line plan would send some customers across the water on a trip more than three times as long, which the road-based zones avoid.

1. Problem Statement

A delivery company runs several warehouses in and around **Kalyani, West Bengal**, and serves customers from Chakdaha in the north to Naihati in the south, on both banks of the Hooghly river. When the list of stops grows too large to inspect by eye, five questions become hard to answer by hand.

1. Which warehouse should serve which customers?
2. Which stops are close enough to be worth grouping?
3. Which customers lie inside a given area, such as a planned delivery zone?
4. How far does the network reach, and which sites are worth keeping?
5. A product is needed at a few places. Which warehouse should send it, and which way should the delivery rider go?

Comparing every point with every other point takes quadratic time, which becomes impractical as the number of stops grows. Each question is a standard algorithmic problem, so each one is handed to the algorithm built for it. The results feed into a single dashboard drawn on the real street map.

Distances follow the **real road network**. In this region a straight-line model has a visible flaw, the Hooghly river. A customer on one bank may find that the nearest warehouse "as the crow flies" is on the other bank, and that warehouse can only reach them over a road bridge or by ferry. The app therefore measures every warehouse-to-customer distance along the OpenStreetMap roads with Dijkstra's algorithm, and §7 shows how wrong the straight-line answers would have been.

This is a **decision-support tool**, not a navigation app. It plans one rider's round on typical road speeds, but it does not model live traffic, fuel, a fleet of vehicles or driver shifts.

2. Input Model

Every warehouse and customer is a point `Point(id, x, y, kind)` with `kind ∈ {warehouse, customer}`. The geometric algorithms only read `x` and `y`, which are kilometres on a flat local grid. Each point also stores the real place behind it (`name`, `lat`, `lon`, OSM `category`) for display, along with a short label such as `W1` or `C12`.

The study area. The area is the convex hull of five town centres, Kalyani, Naihati, Chakdaha, Mogra and Haringhata, with a 2.5 km buffer. It covers **401 km²**, about 25 × 26 km. Each town's coordinates come from an OpenStreetMap (OSM) node, except Kalyani's, which use the centroid of the *Kalyani City* outline (OSM way 1094723487).

The Hooghly divides the area into two banks.

- **West bank (80 km²).** Mogra, Bansberia and Hugli-Chuchura.
- **East bank (321 km²).** Everything else, including Kalyani, Naihati, Halisahar, Kanchrapara, Haringhata and Chakdaha. Chakdaha lies on the east bank because the river curves around Kalyani's north.

The river's main channel runs for 27.5 km through the area.

The points are real places. The October 2026 extract holds **9,422 usable places** inside the area. Of these, 571 are named places such as shops, banks, schools, clinics, offices and college buildings, and 8,851 are buildings, mostly homes, taken from the OSM building footprints. An unnamed building is named from its address, otherwise from the nearest named road within 300 m ("Home near SH1"), otherwise from its town ("Home in Kalyani"). Of all the places, 2,360 are on the west bank and 7,062 on the east bank, and 52 are

tagged as warehouse candidates, such as markets, supermarkets, post offices, warehouses and industrial land.

Latitude and longitude are projected to a flat km grid. An equirectangular projection is anchored at the study area's south-west corner (lat_0 = 22.864°, lon_0 = 88.349°).

```
x = (lon - lon₀) · 111.320 · cos(lat₀)     y = (lat - lat₀) · 110.574
```

So 1 unit is 1 km, and every distance can be reported in metres. The grid stays accurate at this scale. Across all 157,880 pairs of named places at least 0.5 km apart, the flat-grid distance stays within 0.2 % of the true distance on the WGS84 ellipsoid, with a worst case of 0.15 %. The unit tests compare a 9 km diagonal with the haversine great-circle distance and check that it meets the same tolerance.

Sampling is seeded. `sample_network(region, m, n, seed)` uses Python's `random.Random(seed)` to pick the network, so the same seed always gives the same network.

- **Warehouses** are drawn from the 52 candidates first, then from any place. They must be at least `0.55 · √(area / m)` km apart (4.92 km for m = 5). This stops two hubs from sitting next to each other and avoids degenerate service zones. If the area runs out of far-apart places, the minimum gap shrinks by 20 % and the search repeats.
- **Customers** are a uniform random sample of the remaining places. Places at exactly the same spot, to about 1 m, are skipped, so no two customers coincide.

The defaults are seed 42, 5 warehouses, 150 customers and a two-wheeler. The sidebar allows 2 to 8 warehouses and 10 to 1,000 customers. The limit of 8 lets each zone keep its own colour from a fixed palette that was checked for colour blindness.

CSV upload. Users can also upload any network as a CSV with the columns `type, lat, lon` and an optional `name`, where `type` is `warehouse` or `customer`. A file needs at least 1 warehouse and 2 customers, and at most 8 warehouses. The flat grid is then anchored at the uploaded points' own south-west corner, so the tool works for any city, and the zones are clipped to a buffered convex hull of the points. Uploaded networks have no river and no road network, so their distances are straight lines and the River crossing and Route planner pages are skipped. The Data page exports the current network as a CSV with these columns plus `label` and `category`, so a downloaded file can be edited and uploaded again.

2.1 Data pipeline

The OSM extract is downloaded once by `scripts/fetch_osm.py` and saved in the project. The algorithms therefore run offline, and only the background street-map tiles need internet access.

1. **Define the area.** The script builds the padded convex hull of the five anchor towns.
2. **Fetch.** Queries go to the Overpass API for the following data.
    - Every named feature inside the area, plus warehouses and industrial land
    - Every building inside the area, as its centre point with its tags
    - Every drivable road, from motorways down to residential streets and service roads, with all its nodes
    - The ways of the OSM river relation "Hooghly" near the area
    - The road bridges, ferry routes and ferry terminals near the area
    - The Kalyani City outline, which is drawn on the map for context

Overpass is often overloaded, so the script tries several mirrors and retries with exponential back-off. It downloads only the raw files that are missing, and the `--refresh` option downloads them all again.

3. **Keep the raw responses.** The untouched JSON is saved in `data/2026-10/raw/`, with the two large ones (roads and buildings) gzipped. Each month has its own folder, and older folders are never edited.
4. **Clean the places.** Each feature is classified by its main OSM tag, such as amenity, shop, office or healthcare. Roads, rivers, rail lines, parks, parking and plain residential land are skipped, and so are buildings that are not delivery points, such as sheds, garages and water tanks. Duplicates are removed when a place is mapped both as a node and as a building. City and town nodes become map labels (10 towns).
5. **Build the road graph.** Every node of a drivable way becomes a graph node, and every pair of consecutive nodes becomes an edge with its length, road class, name and bridge flag.
6. **Build the river.** The relation's pieces are joined into one main channel. Side channels that loop around islands are dropped, so the line never crosses itself.
7. **Build the crossings (3).**
    - **Ishwar Gupta Setu.** The road bridge between Kalyani and Bansberia, 1.04 km long.
    - **Naihati Ferry Ghat.** Two OSM ferry terminals on opposite banks, paired, 0.82 km apart.
    - **Halisahar ferry ghat.** Only one terminal is mapped, so its far landing is estimated by mirroring it across the channel, 0.52 km away.

    Rail-only bridges (Jubilee and Sampreeti, Naihati to Bandel) are left out because delivery vehicles cannot use them. Ghats the team knows locally can be added by hand in `data/manual/ferries.csv`.
8. **Save.** The outputs are `region.geojson`, `region_places.csv`, `roads.npz`, `towns.csv`, `river.geojson` and `crossings.csv`. The app always loads the newest complete month.

2.2 Road network

The road graph has **121,678 nodes and 129,034 edges, 3,291 km of road** in all. Residential streets make up 2,028 km, followed by unclassified roads (372 km), tertiary (306 km), primary (175 km), service roads (164 km), secondary (104 km), trunk (91 km) and the Kalyani to Barrackpur expressway (42 km). Of the nodes, 99.4 % form one connected network. The graph is stored as compressed NumPy arrays and loads in 0.08 s.

- **Two vehicles.** A **delivery van** may use road bridges only. A **two-wheeler** may also take the two ferries, each modelled as one extra edge between its landings with a 10 minute average wait and 8 km/h on the water.
- **Two weights per edge.** Each edge has a length in km and a travel time in minutes. The time uses a typical speed for the road class and vehicle, for example 35 km/h on a primary road and 22 km/h on a residential street for a two-wheeler, and 55 km/h on the expressway and 18 km/h on a residential street for a van.
- **Snapping.** Every place is tied to its nearest road node with a KD-tree over the road nodes (SciPy's `cKDTree`). For the default customers the gap is 20 m at the median and 86 m at most, and it is added as a walk to the door.
- **Shortest paths.** Dijkstra's algorithm (SciPy's `csgraph.dijkstra`) runs from every warehouse at once and gives the road distance from each warehouse to all 121,678 nodes in 43 ms.

3. Algorithms Used

#	Algorithm	Question answered	Technique	Complexity
1	Convex hull (Andrew's monotone chain, a Graham-scan variant)	How far does the network reach?	Sort the points by (x, y), then build the lower and upper chains, popping any point that makes a non-left turn	O(n log n)
2	Closest pair (divide and conquer), region-wide and per zone	Which two stops are nearest each other?	Split at the x-median, recurse on both halves, then scan only the thin strip around the dividing line for a closer cross-pair	O(n log n), against $O(n^2)$ for brute force
3	KD-tree + range search	Which customers lie inside this rectangle?	Build a binary tree over the customers with alternating x and y median splits. A range query skips whole subtrees that cannot meet the query box	Build O(n log n), query $O(\sqrt{n} + k)$
4	Voronoi diagram by road distance	Which warehouse is nearest to each part of the area?	Dijkstra from every warehouse. Each customer and each 150 m square of the area goes to the warehouse nearest by road	$O(m \cdot E \log V)$ for the distances, then O(m) per customer or square
5	Greedy set cover (optional extension)	Which few warehouses cover everyone?	Each warehouse covers the customers within a road distance. Repeatedly keep the warehouse that covers the most still-uncovered customers	The exact optimum is NP-hard, and greedy stays within $H(n) \approx \ln n$ of it
6	Route planning (Dijkstra, Held-Karp, nearest neighbour + 2-opt)	Which way should the rider go?	Travel times between all stops by Dijkstra, then the visiting order by dynamic programming (exact) or a local-search heuristic	Held-Karp $O(2^k k^2)$ for k stops, 2-opt $O(k^2)$ per pass
+	River check (real-world extension, §7)	Where does straight-line planning break?	Find each customer whose straight-line nearest warehouse is on the other bank and measure the real road trip	One Dijkstra per warehouse, then O(m) per customer

Here n is the number of customers, m the number of warehouses, k the number of stops on a route (or of points a query reports), V and E the road nodes and edges.

Each algorithm is also shown in the app in plain words and in pseudocode, and the Walkthrough highlights the pseudocode line that the current step is running. The same pseudocode is given below.

3.1 Convex Hull with Andrew's Monotone Chain

The hull uses **Andrew's monotone chain**, a variant of Graham's scan. Graham's scan sorts the points by polar angle around the lowest point. Andrew's version sorts them by `(x, y)` instead and builds the hull in two halves, which avoids angle comparisons and handles ties simply.

The customers are sorted by `(x, y)`, with duplicates removed. The **lower chain** is built by scanning them from west to east. Each new point is pushed onto a stack, but first the top of the stack is popped for as long as the last two stack points and the new point fail to make a left (counter-clockwise) turn. The turn is tested with the sign of a cross product. The **upper chain** is built the same way, from east to west. Joining the two chains gives the hull in counter-clockwise order, starting from the westmost stop. Collinear boundary points are dropped. Each point is pushed and popped at most once per chain, so the scan takes linear time after the O(n log n) sort.

```
ConvexHull(P):
  sort P by x (ties by y)
  lower ← empty chain
  for each p in P, west to east:
    while lower has ≥ 2 points and last two + p turn right:
      remove the last point          ▷ it makes a dent
    add p to lower
  upper ← the same scan, east to west
  return lower + upper (shared ends once)
```

The area the hull encloses is reported as the network's **reach**. The hull is geometric on purpose, because it describes where the customers are, not how to drive to them.

3.2 Closest Pair by Divide and Conquer

The customers are sorted by x and split at the median into two halves. Each half is solved recursively, and three or fewer points are solved by brute force. Let δ be the smaller of the best distances in the two halves. Any closer pair must have one point on each side, with both points within δ of the dividing vertical line. So only the strip of width 2δ around that line needs to be checked again. The strip's points are taken in y order, and each point is compared only with the points above it until the y-gap reaches δ. A packing argument limits this to a constant number of comparisons per point.

No level sorts anything. The points are sorted by x once at the start, and each recursive call returns its points sorted by y. That order is built by merging the two halves' y-orders in linear time, as in merge sort. The strip is filtered from the merged list, which gives $T(n) = 2T(n/2) + O(n) = O(n \log n)$. Comparing every pair would take $O(n^2)$.

```
ClosestPair(P):
  sort P by x once;  return Solve(P)
Solve(P):
  if |P| ≤ 3: compare every pair, return the best
  split P at the middle x into L and R   ▷ divide
  bestL ← Solve(L);  bestR ← Solve(R)      ▷ conquer
  δ ← the smaller of bestL, bestR
  merge L and R by y                    ▷ linear, like merge sort
  S ← points closer than δ to the cut, by y
  for each p in S, check the next points while Δy < δ:
    if closer than δ: δ ← that distance, keep the pair
  return the best pair
```

The dashboard runs the algorithm on all customers to find the two nearest stops in the whole region. It also runs it **inside each warehouse's zone**, giving each warehouse its own pair of stops worth grouping into one delivery, and it reports the road distance between the two stops of the region-wide pair as well.

3.3 KD-Tree and Range Search

A KD-tree is built by recursively splitting the customer set at the median x (at even depth) or the median y (at odd depth). This keeps the tree balanced, with a depth of about $\log_2 n$. The points are sorted once by x and once by y. Each node takes the median of the order for its axis and splits both orders in one linear pass by

comparing ranks with the median's rank. Both orders stay sorted without any re-sorting, which gives T(n) = 2T(n/2) + O(n) = O(n log n).

A rectangle range query starts at the root and recurses down the tree. At each node it reports the node's point if it lies inside the box, then visits a child only if the box reaches that side of the splitting line. Whole subtrees are skipped when the box cannot touch their part of the area. A query therefore visits only the points near the box and the ones it reports. For a balanced 2D tree this costs O(√n + k), instead of a scan of every customer.

```
Build(P, depth):
  if P is empty: return nothing
  axis ← x if depth is even, else y
  m ← the median point of P on axis      ▷ from the presorted lists
  node ← m
  node.left  ← Build(points before m on axis, depth + 1)
  node.right ← Build(points after m on axis, depth + 1)
  return node

Search(node, box):
  if node is empty: return
  if node.point is inside box: report it
  v ← node.point's coordinate on node.axis
  if box.low ≤ v:  Search(node.left, box)
  if box.high ≥ v: Search(node.right, box)
  ▷ a side the box cannot reach is skipped, whole subtree
```

The same idea, through SciPy's KD-tree, snaps every place to its nearest road node (§2.2).

3.4 Voronoi Diagram by Road Distance

Each warehouse owns a zone of the area, which holds every spot that is closer to that warehouse than to any other. Here closer means **along the roads**. This is a *network Voronoi diagram*, the Voronoi diagram under shortest-path distance instead of the straight line. It respects the river and the road layout without any special case, because a spot across the water is simply far away by road.

- **Distances.** Dijkstra runs once from every warehouse over the road graph for the chosen vehicle. Adding the walk from the road to the door gives the door-to-door road distance from every warehouse to every customer.
- **Assignment.** Each customer is compared with every warehouse, and the nearest by road wins. This takes O(m) per customer and is exact by the definition of a Voronoi cell.
- **Zones on the map.** The area is covered with 150 m squares (18,320 of them), each snapped to its nearest road node and given to the warehouse nearest by road. The squares of each warehouse are merged into one shape, first along each row and then with a polygon union, and the shared borders of neighbouring zones are smoothed together (coverage simplification), so the zones never overlap or leave gaps. All five zones are built in 51 ms, Dijkstra included.

```
Assign(customer c, warehouses W):
  best ← none;  bestDist ← ∞
  for each warehouse w in W:
    d ← road distance from w to c   ▷ Dijkstra from w
    if d < bestDist: best ← w;  bestDist ← d
  return best                ▷ c lies in best's cell
ServiceZones(W):
  for each warehouse w: D[w] ← Dijkstra(w) over the roads
  for each 150 m square s of the area:
    owner(s) ← the w with the smallest D[w] at s's road point
  merge the squares of each owner into its zone
```

The **classic straight-line Voronoi diagram** is still computed alongside. It can be drawn over the road zones on the Network map as dashed outlines, the Walkthrough can trace a customer's assignment either way, and it is the zoning for an uploaded CSV, which has no roads. Its cells come from `scipy.spatial.Voronoi`, which wraps Qhull and builds the dual of the Delaunay triangulation in O(m log m) expected time, and are clipped to the area with shapely. Comparing the two on the map shows exactly where the river and the road layout move a border.

3.5 Greedy Set Cover

The warehouses are the candidate sites, and each one covers the customers it can reach within a chosen distance **along the roads**, using the same Dijkstra distances as the zones. Finding the true minimum number of sites that covers every customer is an NP-hard set-cover problem, so a greedy heuristic is used instead. It repeatedly keeps the candidate that covers the most still-uncovered customers, until every customer is covered or no candidate can add anyone new. This greedy rule is proven to stay within a factor `H(n) = 1 + 1/2 + ... + 1/n ≈ ln n` of the true optimum. The map shades the part of the area that each kept warehouse reaches by road, using the same 150 m squares as the zones.

```
GreedyCover(customers U, sites S, distance r):
  for each site s: cover(s) ← customers within r of s by road
  covered ← ∅;  chosen ← []
  while covered ≠ U:
    s ← the site with the most customers in cover(s) - covered
    if s adds nobody new: stop
    add s to chosen;  covered ← covered ∪ cover(s)
  return chosen
```

The set-cover example from the project overview, with four sites, nine customers and the answer W2 then W4, is one of the unit tests.

3.6 Route Planning

The Route planner answers the delivery question directly. The user picks a product and one or more places that need it. Each warehouse stocks a fixed random selection of eight products (seeded, so a network always has the same stock), and every product is stocked somewhere.

1. Snap the stops and the warehouses that stock the product to road nodes.

2. Run Dijkstra on travel time from each of those warehouses and each stop. This gives the time between every pair, including the river crossing the vehicle may use.
3. Add 3 minutes per stop for the hand-over, plus the walk from the road to the door and back.
4. Choose the visiting order. Up to 10 stops the **Held-Karp** dynamic program gives the exact best order. It stores, for every set S of stops and every last stop j, the quickest way to leave the warehouse, visit all of S and end at j. Above 10 stops its $O(2^k k^2)$ cost grows too fast, so a **nearest-neighbour** tour is built and then improved by **2-opt**, which reverses any stretch of the order whenever that shortens the round.
5. Try every warehouse that stocks the product and keep the fastest round. Each leg's path is recovered from Dijkstra's predecessor array and drawn on the map.

```
PlanRoute(product, stops):
  snap stops and warehouses to road nodes      ▷ KD-tree
  T ← Dijkstra times from every warehouse and stop
  for each warehouse w that stocks the product:
    cost[a][b] ← T[a][b] + hand-over + walk at b
    if stops ≤ 10: order ← HeldKarp(cost)        ▷ exact
    else: order ← NearestNeighbour(cost), then 2-opt
  keep the warehouse and order with the least time
HeldKarp(cost):
  best[S][j] ← quickest way from w through set S, ending at j
  best[{j}][j] ← cost[w][j]
  best[S][j] ← min over i in S-{j} of best[S-{j}][i] + cost[i][j]
  answer ← min over j of best[all][j] (+ cost[j][w] to return)
```

4. How the Algorithms Combine

- **Dijkstra + everything that measures distance.** One Dijkstra per warehouse feeds the Voronoi zones, the set cover, the river check and the distances on the Data page, so every answer about "how far" follows the roads.
- **Voronoi + KD-tree.** Voronoi decides which warehouse owns each customer. The KD-tree then pulls out the customers inside any rectangle on the map, and their warehouse labels come along with them. One query therefore answers both "who is here" and "who serves them".
- **Voronoi + closest pair.** Closest pair runs inside each warehouse's zone to find the stops worth grouping into a shared delivery.
- **Convex hull + everything.** The hull marks the outer edge of the customer set and shows how far the network reaches compared with the whole area. Its corners are the outlying stops, which are also the stops most likely to fall outside a coverage distance.
- **Set cover + Voronoi.** Voronoi assigns each customer to exactly one warehouse. Set cover asks a different question, which warehouses are needed at all for a given reach.
- **Route planner + KD-tree + Dijkstra.** The KD-tree ties every stop to the road, Dijkstra gives the times between stops, and Held-Karp or 2-opt puts them in order.

- **Voronoi + river check.** The river check compares the road-based zones with what a straight-line plan would have chosen, and measures the difference.

5. Dashboard

The dashboard is a Streamlit app (`app.py` and `ui/`) with seven pages in a top navigation bar. It runs online at smart-delivery-network.streamlit.app on Streamlit Community Cloud. The maps use deck.gl through pydeck on CARTO's light "Positron" basemap, which is built from OpenStreetMap data. The app uses a single light theme. Every page has a header with the project name, the course and summary chips, and the Network map and How it works pages also list the team.

The **sidebar** chooses the data source (*Kalyani region* or *Upload CSV*), the number of warehouses (2 to 8) and customers (10 to 1,000), the random seed and the delivery vehicle (*Two-wheeler* or *Delivery van*). A *Shuffle network* button picks a new seed. The roads are used for the calculations only, so the maps stay uncluttered, and a *Show road paths on maps* switch draws the street-by-street paths on the River crossing and Walkthrough maps when wanted. The sidebar also shows the course, the team and the OSM and CARTO credits.

1. **Network map.** The headline cards show the following.
   - Service zones and the busiest zone
   - Delivery stops and their average road distance to their hub
   - Network reach, with the hull area and the number of boundary stops
   - The closest pair
   - The customers in the search area
   - The customers whose straight-line hub lies across the Hooghly

   The map shows the Voronoi zones by road, coloured by warehouse, and a *Classic Voronoi* layer can add the straight-line cells as dashed outlines for comparison. It also shows the convex hull, the KD-tree search area with its hits ringed, the closest pair across the region as a thick line and the per-zone pairs as thin lines. The river with its bridge and ferries and the Kalyani City outline add context, and each layer can be switched off. Clicking any dot shows details about that place. A warehouse shows how many customers it serves, and a customer shows its three nearest warehouses by road. Either can become the centre of the search area.

   A side panel has three tabs.

   - **Search area.** The centre can be Kalyani or any place. Width and height range from 0.5 to 10 km, with a default of 3.0 × 2.4 km. Hits are listed from nearest to farthest.
   - **Zones.** The number of customers and the average and farthest road distance for each zone.
   - **Closest stops.** The closest pair across the region and in each zone.
2. **Route planner.** A product, the places that need it (picked from a searchable list, five at random, or by clicking customers on the map), the start warehouse (the best one with stock, or a fixed one), whether the rider returns, and the dispatch time. The cards show the number of stops, the start warehouse, the road distance against the straight line, the total time, the time back at base and whether the order is exact. The map draws the route with numbered stops, and the itinerary lists every leg with its distance, time, roads, ferry if any, and arrival time. A *live tracking* slider moves the rider along the route and marks the stops delivered so far.

3. **River crossing.** This page runs the river check of §7. A switch chooses between *Delivery van*, which may use road bridges only, and *Two-wheeler*, which may use bridges and ferries. The cards show the following.
   - The west-bank customers
   - How many have their straight-line hub across the river
   - The average straight line and real road trip
   - The extra distance
   - How many have a nearer hub by road

   The map draws each affected customer's straight line next to its real trip over the bridge or ferry. A side list ranks the affected customers by extra distance and lists the three crossings.
4. **Coverage planner.** Greedy set cover with a road-distance slider from 1 to 10 km and a default of 4 km. The cards show how many warehouses are kept, the share of customers covered, how many are not covered, the number of rounds and the H(n) guarantee. The map shades the area each kept warehouse reaches by road, shows the warehouses that could close in grey, and rings every uncovered customer in red. A round-by-round timeline lists each pick, how many new customers it covered and the running total.
5. **Walkthrough.** This page steps through every decision of five algorithms on the customers currently on the map.
   - **Convex hull.** The chain being built and each popped point.
   - **Closest pair.** Each split line, base case and strip of width 2δ.
   - **KD-tree build.** Each median split.
   - **KD-tree search.** Each visited node and skipped subtree.
   - **Voronoi assignment.** One customer compared with each warehouse in turn, by road distance or, with a switch, in a straight line as in the classic diagram.

   Arrows and a slider move between steps. Beside each step the algorithm's pseudocode is shown with the line being run highlighted, and the algorithm is explained in plain words under the map. The traces come from instrumented copies of the algorithms in `src/traces.py`. These copies are kept apart from the tested core code so the teaching output cannot change a result, and a test checks that the traces agree with the core algorithms. The Coverage planner and the Route planner carry the same plain-words and pseudocode explanation.
6. **Data.** The full point set as a table with label, name, type, OSM category, zone, bank, road and straight-line distance to the hub, x and y in km, and latitude and longitude. The page also has a CSV download, a bar chart of customers per zone and a chart of the kinds of places.
7. **How it works.** One card per algorithm, one for the river check and one for the road network and route planner, each with its question and complexity. The page then explains the OSM pipeline and sets out the scope and limits.

6. Worked Example (default network)

The input is the October 2026 OSM extract with the app's defaults, **seed 42, 5 warehouses, 150 customers and a two-wheeler**, the default search area (3.0 × 2.4 km around Kalyani City) and the default 4 km coverage distance. Every number below can be reproduced with `python report/worked_example.py`, which calls the same code as the app.

Warehouses chosen by the seeded sampler (no two are closer than 6.64 km)

Label	OSM place	Category	Bank	x (km)	y (km)
W1	More (Kalyani)	supermarket	east	9.00	12.66
W2	Naihati Post Office	post office	east	7.21	2.35
W3	Industrial site	industrial	west	3.83	8.06
W4	Flipkart Warehouse	warehouse	east	19.60	13.27
W5	Chakdaha Post Office	post office	east	18.51	23.72

Of the 150 customers, 133 are homes and the rest are named places such as a school, a college, a hospital and a bank. 42 customers are on the west bank.

Results

Step	Input	Output
Voronoi by road	5 warehouses, 150 customers, the road graph	5 zones that tile the whole 400.78 km². They hold W1 72, W2 8, W3 42, W4 25 and W5 3 customers. The average road distance to the assigned hub is 3.65 km, against 2.63 km in a straight line. 4 customers have a different warehouse than a straight-line plan would give them.
Convex hull	150 customers	**10 boundary stops**, enclosing 217.9 km² (54 % of the study area)
Closest pair (region-wide)	150 customers	**C94 and C136, two homes in Kalyani, 21 m apart** (84 m by road). Brute force would compare 11,175 pairs.
Closest pair (per zone)	Each zone's customers	W1 (C94 and C136, 21 m) · W4 (C56 and C64, 37 m) · W3 (C4 and C67, 40 m) · W2 (C131 and C27, 84 m) · W5 (C24 and C62, 130 m)
KD-tree range query	The default search area of 3.0 × 2.4 km centred on Kalyani City's centroid (x 9.32 to 12.32 km, y 11.22 to 13.62 km)	**35 customers**, all in zone W1. The search visited 56 of the 150 tree nodes and skipped 13 subtrees, and a brute-force filter returns the same 35.
Greedy set cover	5 candidate warehouses, 4 km by road	**4 rounds**, keeping W1 (+70, 70/150), W3 (+32, 102/150), W2 (+6, 108/150) and W5 (+3, 111/150). **W4 could close**, and **39 customers** are more than 4 km by road from every warehouse.

Zones by road against straight-line cells

Zone	Customers	Area by road	Straight-line cell	Mean road distance	Farthest
W1 More	72	100.6 km²	119.1 km²	2.70 km	5.17 km
W2 Naihati Post Office	8	52.1 km²	53.8 km²	3.16 km	5.69 km

Zone	Customers	Area by road	Straight-line cell	Mean road distance	Farthest
W3 Industrial site	42	79.2 km²	46.1 km²	2.79 km	7.91 km
W4 Flipkart Warehouse	25	86.3 km²	117.1 km²	8.27 km	10.36 km
W5 Chakdaha Post Office	3	82.5 km²	64.7 km²	1.50 km	1.58 km

Step-by-step traces. These counts show the work each algorithm does on this input.

- **Convex hull.** 588 steps (300 pushes, 288 pops).
- **Closest pair.** 253 steps (63 divides, 64 base cases, 63 combines and 63 strip checks).
- **KD-tree build.** 150 median splits over 8 levels.
- **KD-tree search.** 69 steps (56 visits, 13 skipped subtrees).
- **Voronoi trace.** Customer C1 (a home in Naihati) is 11.56 km by road from W1, 1.56 km from W2, 7.13 km from W3, 19.98 km from W4 and 26.78 km from W5, so it joins zone W2.

What the numbers say

- **The road zones follow the river.** By road, W3 owns 90 % of the west bank and its zone grows from 46 km² to 79 km², while W1 and W4 lose the parts of their straight-line cells that lie across the water or behind poor roads. No special river rule is needed, the shortest paths take care of it.
- **Roads are longer than straight lines, but not uniformly.** The road distance to the straight-line hub is 1.36 times the straight line at the median and 1.56 times on average, but up to 9.4 times for a customer whose straight-line hub is across the river.
- **W1 (More, a supermarket in Kalyani) is the busiest hub**, with 72 of the 150 customers. Kalyani holds the densest mapping, and the whole default search area falls inside W1's zone.
- **W4 is the weak site.** Its 25 customers are 8.27 km away by road on average, so at 4 km it reaches nobody whom another warehouse does not already reach, and it could close. With straight-line circles all five warehouses would have been kept (124 of 150 covered), which overstates W4's real reach.
- **The hull covers 54 % of the study area**, so the customers are spread over the whole region rather than only Kalyani.
- **The closest pair is two neighbouring homes 21 m apart**, 84 m by road. One delivery could serve both.

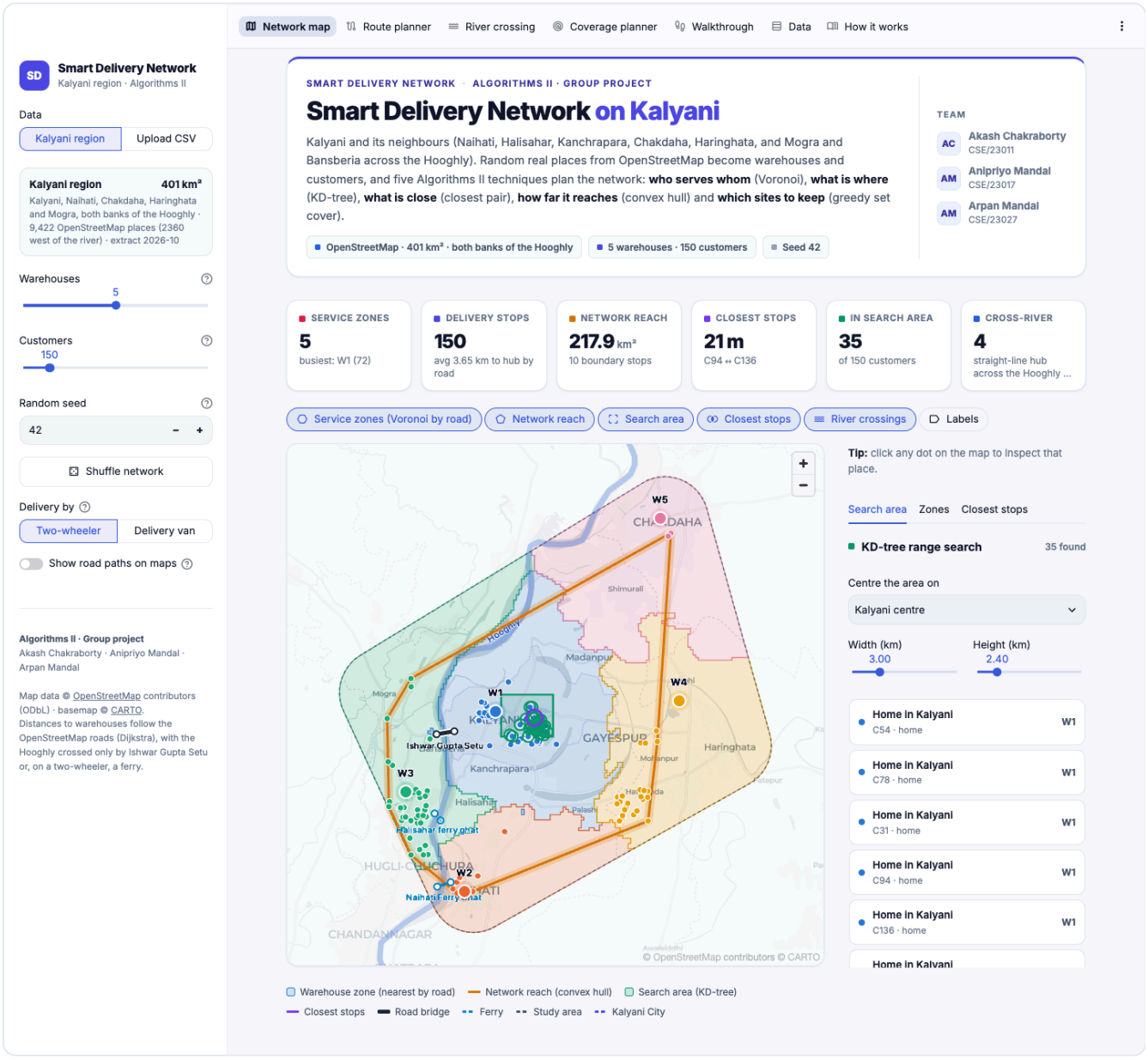


Figure 1. Network map. The Network map page for the default network, with the Voronoi service zones computed by road distance and coloured by warehouse, the convex hull (orange), the KD-tree search area around Kalyani with its 35 hits ringed (green), the closest stops across the region and per zone (purple), and the Hooghly with Ishwar Gupta Setu and the two ferry ghats. W3's zone (green) covers almost the whole west bank. The cards above the map report 5 zones, 150 stops with an average of 3.65 km to the hub by road, a 217.9 km² reach with 10 boundary stops, a 21 m closest pair, 35 customers in the search area and 4 customers whose straight-line hub is across the river. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

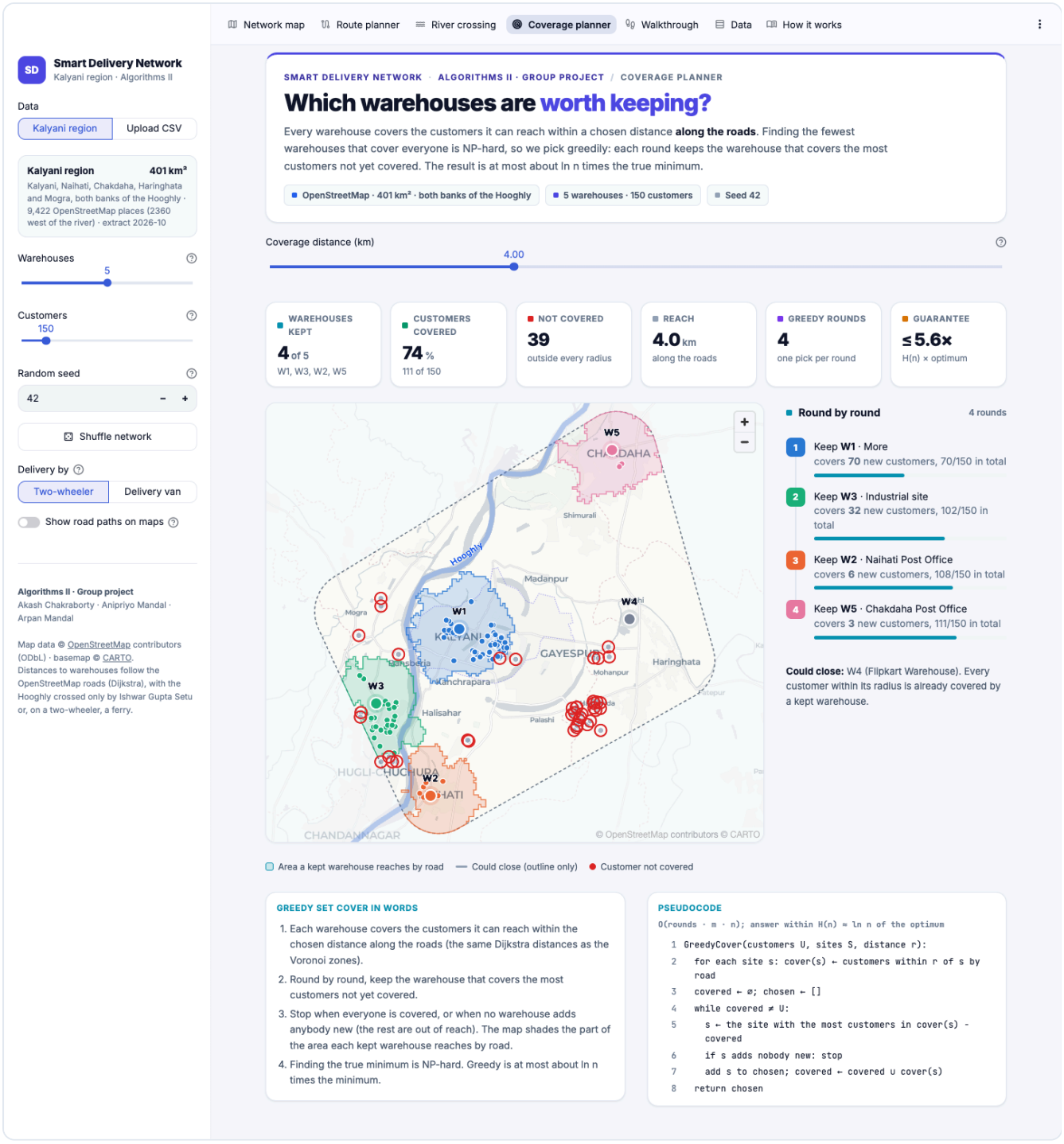


Figure 2. Coverage planner. The Coverage planner at the default 4 km road distance. The shaded areas are what each kept warehouse reaches along the roads, and the 39 uncovered customers are ringed in red. The timeline lists the four greedy rounds (W1, W3, W2, W5) up to 111 of 150, and W4 could close. Under the map the algorithm is explained in plain words and in pseudocode. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

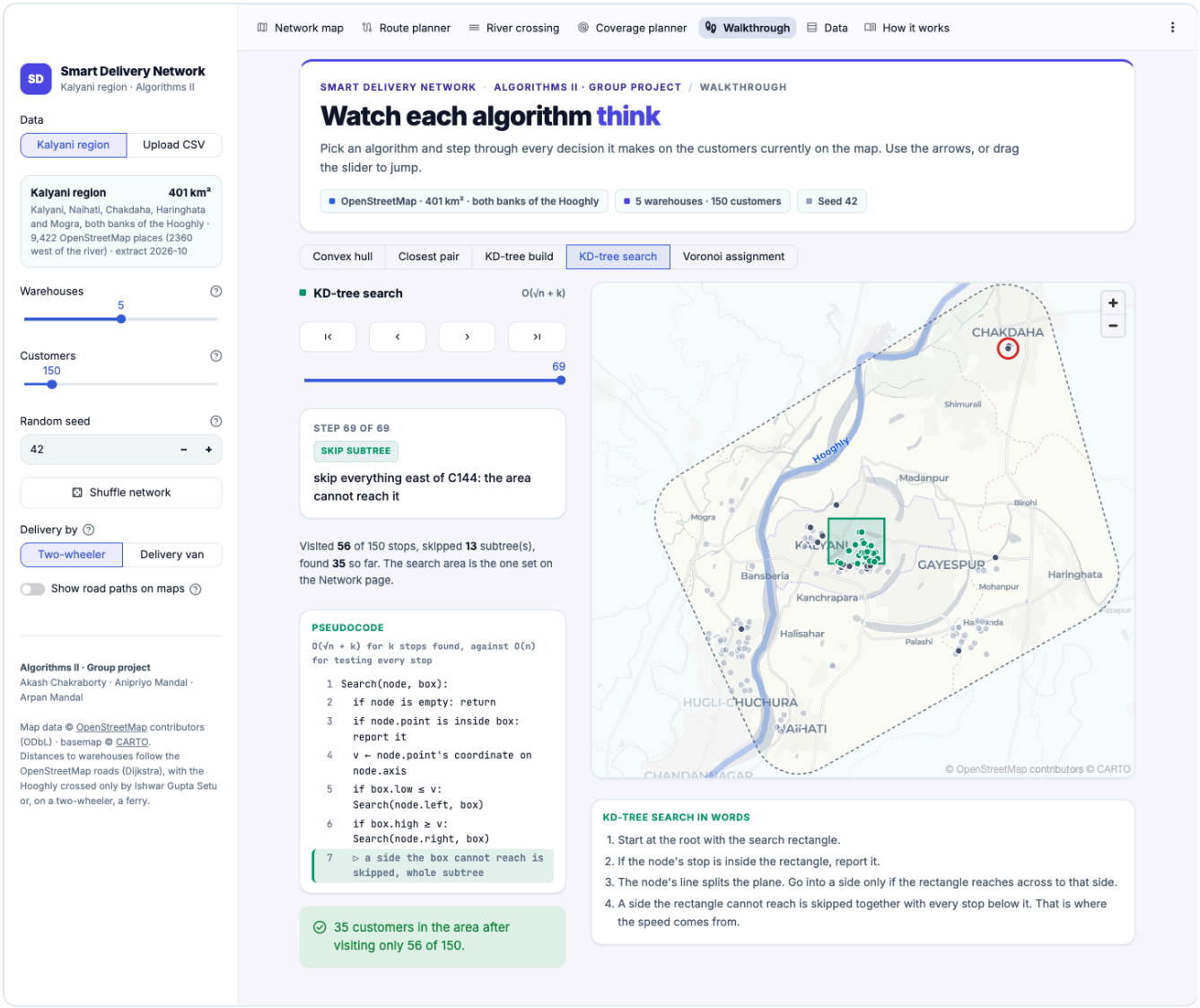


Figure 3. Walkthrough of the KD-tree search. The Walkthrough page at step 69 of 69 of the KD-tree search, with the default search area around Kalyani (green box), the 35 customers found inside it (green), the nodes the search visited (dark grey) and the stops it never had to look at (light grey). The pseudocode card highlights the line the step is running (skip a subtree the box cannot reach), and the algorithm is described in plain words under the map. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

7. Crossing the Hooghly, a Real-World Issue

A textbook Voronoi diagram assigns each customer to the warehouse that is nearest **in a straight line**. Across a river this can be badly wrong, because the trip has to use a crossing. The app avoids the problem by measuring every distance along the roads (§3.4), and the River crossing page shows what a straight-line plan would have got wrong.

Model

- **Banks.** The river's main channel is extended at both ends past the study area and used to cut the area into two pieces with shapely. The piece whose centroid lies further west is the west bank, and a point's bank is the piece that contains it. Points that fall in neither piece are decided by a cross-product side test against the nearest stretch of the channel.
- **Trips.** A trip is the shortest road path for the vehicle. A **delivery van** can only cross on Ishwar Gupta Setu, which is part of the road graph. A **two-wheeler** may also take the two ferries, which are extra edges in its graph.
- **Check.** For every customer whose straight-line nearest warehouse is on the other bank, the check records the straight-line distance, the real road trip, the crossing used and the warehouse that is really nearest by road.

Results on the default network (42 customers and one warehouse, W3, are on the west bank)

	Delivery van (road bridges only)	Two-wheeler (bridges + ferries)
Customers whose straight-line hub is across the river	4	4
Average straight-line distance (what a straight-line plan assumes)	4.12 km	4.12 km
Average real road trip	14.78 km	7.57 km
Extra distance per trip	**+10.66 km (+259 %)**	**+3.45 km (+84 %)**
Crossings used	Ishwar Gupta Setu × 4	Ishwar Gupta Setu × 2, Naihati ferry × 2
A nearer warehouse exists by road	4 of 4	4 of 4

- **The worst cases are C10 and C80, two homes near Mahendra Mitra on the west bank.** A straight-line plan gives them to W2 in Naihati, 3 km away across the water. By van the only way goes up to Ishwar Gupta Setu and back down, 20.5 km, while W3 on their own bank is 4.5 to 4.7 km away by road. By two-wheeler the Naihati ferry cuts the trip to 6.1 to 6.2 km, but W3 is still nearer.
- **C58 and C95 near Bansberia** are nearest to W1 in Kalyani in a straight line (5.1 to 5.2 km), but the trip over the bridge is 8.8 to 9.2 km and W3 is 7.4 to 7.9 km away.

The road-based zones of §3.4 already give all four customers to W3, so the app's plan is right where a straight-line plan would be wrong. With only one road bridge along the 27.5 km of river inside the area, a van's real trip to a cross-river hub is more than three times the straight line.

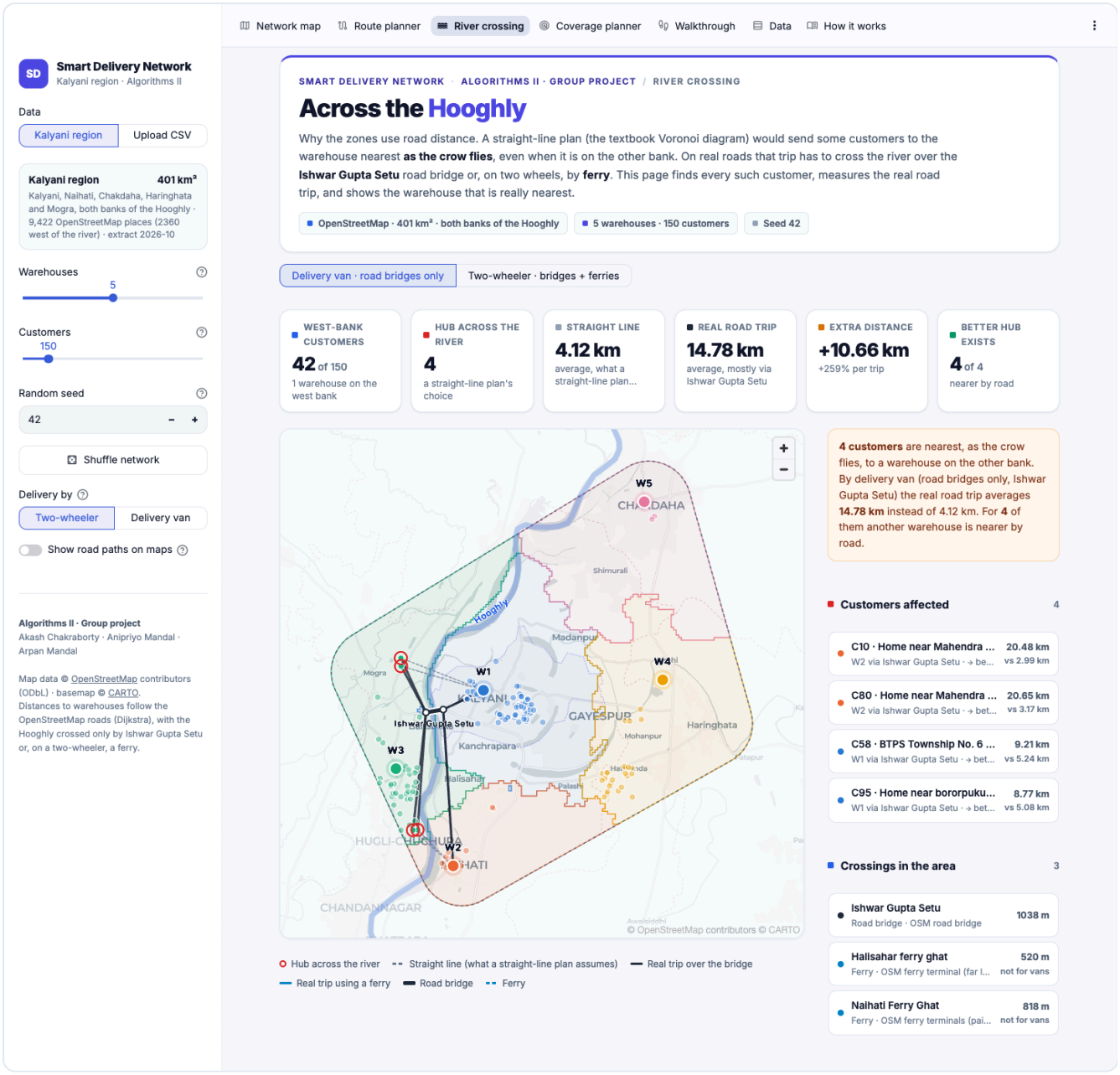


Figure 4. River crossing. The River crossing page in delivery-van mode. The 4 customers whose straight-line hub is across the Hooghly are ringed in red, with their straight lines (dashed) and their real trips over Ishwar Gupta Setu (solid). The cards report 42 west-bank customers, 4 with a straight-line hub across the river, an average of 4.12 km in a straight line against 14.78 km by road (+259 %), and a nearer hub by road for all 4. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

8. Route Planner Example

The Route planner opens with an example on the default network. **Medicines** are needed at five random customers, and the rider returns to the warehouse at the end. Medicines are stocked at W2, W4 and W5, so the planner tries all three as the start.

By two-wheeler the best start is **W2 (Naihati Post Office)**. Held-Karp gives the exact order, **47.3 km and 2 h 00 min** in all (riding, walking and hand-overs), against 29.9 km in a straight line. Planning takes 71 ms.

Leg	To	Road distance	Time	Main roads
1	C71, a home in Hugli-Chuchura	6.9 km	33 min	SH1, Naihati Ferry Ghat
2	C7, Elite Collage Ground	10.1 km	23 min	Grand Trunk Road, STKK Road
3	C58, BTPS Township No. 6 ground	3.7 km	12 min	Grand Trunk Road, SH6
4	C29, a home near Congress Road	9.7 km	22 min	SH6, Grand Trunk Road, Kalyani to Barrackpur Expressway
5	C63, a home in Kalyani	3.6 km	10 min	SH1
6	Back to W2	13.3 km	21 min	Kalyani to Barrackpur Expressway

The rider takes the Naihati ferry to the west bank, works north along the Grand Trunk Road, crosses back on Ishwar Gupta Setu to Kalyani and returns on the expressway. The first leg is short but slow, because it includes the 10 minute wait for the ferry.

By delivery van the same five stops take **61.5 km and 2 h 03 min**. The van cannot use the ferry, so it runs the round the other way and crosses Ishwar Gupta Setu twice. It drives 14 km more but loses only 3 minutes, because it is faster on the expressway and never waits for a ferry.

Exact against heuristic order. On 20 random rounds of 10 stops, with real two-wheeler travel times, nearest neighbour + 2-opt found the optimal order 11 times. On average it was 0.76 % longer than the Held-Karp optimum, and 5.80 % at worst. Held-Karp took 5 ms on average and 2-opt 0.2 ms, so above 10 stops, where Held-Karp's cost doubles with every extra stop, the heuristic is a sensible choice. A 20-stop round is planned in 0.23 s (87.1 km, 3 h 57 min).

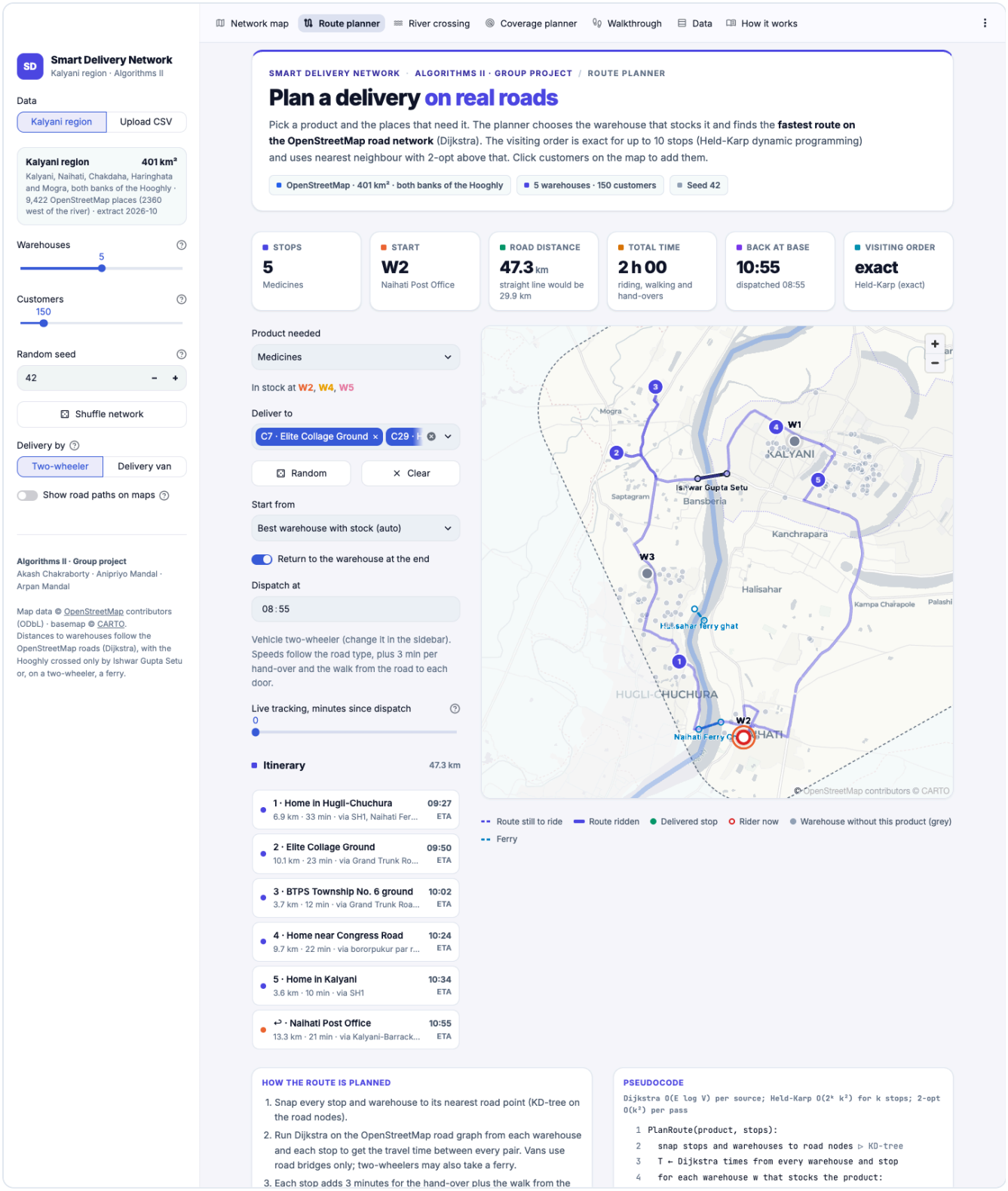


Figure 5. Route planner. The Route planner with its opening example, Medicines from W2 (Naihati Post Office) to five customers by two-wheeler. The numbered stops show the exact Held-Karp order, the dashed line is the route still to ride, and the itinerary on the left gives each leg's distance, time, roads and arrival time. The cards report 5 stops, 47.3 km by road against 29.9 km in a straight line, 2 h 00 min in all and an exact order. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

9. Testing

There are **59 automated tests** in four files, and all of them pass (`python -m pytest tests/ -v`).

`tests/test_algorithms.py` (19 tests) checks each algorithm against a brute-force reference or a known answer.

- Closest pair matches an $O(n^2)$ pairwise scan over five random seeds and many input sizes. It handles duplicate coordinates and passes a small hand-made case. The y-merge is sorted and stable, and a test fails if the recursion ever sorts again after the single initial sort by x, which guards the O(n log n) bound.
- Convex hull keeps only the corners of a square with an interior point, and handles a fully collinear point set as a degenerate case.
- KD-tree range queries match a brute-force filter, also with many duplicate coordinates, and an empty box returns nothing. The tree keeps the split invariant and stays balanced when values tie, and a test fails if the build sorts anything beyond the two initial presorts.
- Greedy set cover covers every coverable customer, reproduces the overview document's worked example (W2 then W4) and stops when no candidate adds anything new.
- The Walkthrough traces agree with the core algorithms for the hull, closest pair and KD-tree search, also on duplicate points, and the Voronoi trace ends at the nearest warehouse.

`tests/test_geo.py` (10 tests) checks the geography and data handling.

- Projecting latitude and longitude to the grid and back gives the original values, and a 9 km diagonal agrees with the haversine distance to within 0.2 %.
- The OSM extract is sensible, with an area of 300 to 500 km^2, at least 400 places, at least 50 of them on the west bank, and all of them inside the area.
- Sampling is reproducible for a fixed seed. It never puts a customer on a warehouse or two customers on the same spot, and it keeps warehouses more than 2 km apart.
- For 1, 2, 5 and 8 warehouses, the straight-line Voronoi cells partition the area, so their areas sum to the area's total, and every customer lies inside the cell of its nearest warehouse (4 tests).
- A network survives a CSV export and re-import, and a CSV with missing columns is rejected.

`tests/test_river.py` (7 tests) checks the river model, mostly on a toy 10 × 10 km square split by a north to south river with one bridge and one ferry.

- The two banks are found, and every crossing is oriented from west to east.
- A trip between two points on the same bank is the straight line.
- A trip across the river uses the best allowed crossing. The two-wheeler takes the ferry, while the van must use the bridge and travels further.
- With no allowed crossing, the trip is impossible (infinite).
- The river check flags only cross-bank assignments and finds the better hub.
- Joining the river pieces gives one simple line past an island.
- On the real data, Mogra is on the west bank, and Kalyani, Naihati, Chakdaha and Haringhata are on the east bank. Ishwar Gupta Setu is a bridge, and every crossing joins the two banks.

`tests/test_roads.py` (23 tests) checks the road network and the route planner.

- Held-Karp gives the same order cost as trying every permutation, on six random instances, both for rounds that return to the warehouse and for open ones (12 tests).
- Nearest neighbour + 2-opt always returns every stop once and is never better than the Held-Karp optimum, which would expose a bug in either (6 tests).
- The planner switches from Held-Karp to the heuristic above 10 stops.
- On the real road graph, no road distance is shorter than the straight line.
- A planned route on the real graph is one continuous path that starts and ends at the warehouse, passes every stop, and adds up to its total distance.
- A van cannot take a ferry and has to go round by the bridge.
- The zones by road partition the study area without overlaps, and each warehouse lies inside its own zone.

10. Discussion

Limitations

- **Typical speeds, not live traffic.** Travel times use one speed per road class and vehicle. There is no traffic, no time of day and no fuel model.
- **The road graph is undirected.** One-way streets and turn restrictions are ignored, and roads missing from OSM are missing from the graph.
- **Zones on the map are drawn on a 150 m grid.** The assignment of each customer is exact, but the drawn border between two zones is accurate to about one square, and spots far from any road (fields, the river itself) are given to the warehouse of their nearest road.
- **The river model is simplified.** The OSM channel geometry is coarse, with vertices up to 1.5 km apart, so places right on the bank can land on the wrong side. Only three crossings are known in OSM for this stretch, the far landing of the Halisahar ferry is estimated, and ferries have a fixed average wait instead of a timetable.
- **One rider, one round.** The Route planner orders the stops of a single trip. It does not split orders between several riders, respect vehicle capacity or delivery time windows.
- **Held-Karp is exponential.** It is used only up to 10 stops, and above that 2-opt gives a good but not guaranteed order.
- **The customers are random real OSM places**, mostly homes from the building footprints, standing in for customers, not real orders. OSM coverage is uneven, with places clustered in Kalyani and sparse in the east and north, so some zones get few customers, as W2 and W5 do in the example above.
- **Greedy set cover** gives an approximate rather than optimal warehouse subset. For instances this small, an exact solver would also be feasible.
- **The hull, closest pair and KD-tree stay geometric.** They describe where the customers are, so straight lines suit them, but the closest pair by road can differ from the closest pair in a straight line.
- **The flat projection is valid only at regional scale.** An uploaded CSV that spans a whole country would distort distances.

Future Scope

- Make the road graph directed, with one-way streets and turn restrictions, and add time-of-day speeds or live traffic.
- Extend the Route planner to several riders with capacities and delivery time windows (the vehicle routing problem), for example with savings or insertion heuristics followed by 2-opt.
- Extend the model from static demand, where all customers must always be served, to **capacitated** facility assignment, where each warehouse has a maximum number of customers it can serve and customers are weighted by expected demand.
- Add real ferry timetables and the ferry ghats the team knows locally to `data/manual/ferries.csv`.
- Draw exact network Voronoi borders along the road edges instead of a grid.
- Implement Fortune's sweep for the straight-line Voronoi diagram and add it to the Walkthrough.

11. Data Sources and Attribution

- **Map data © OpenStreetMap contributors**, available under the Open Database License (ODbL), see openstreetmap.org/copyright. The places, buildings, roads, towns, river, crossings and Kalyani City outline were downloaded through the Overpass API in October 2026 and are stored in `data/2026-10/`.
- **Basemap tiles © CARTO**, see carto.com/attributions. The dashboard's maps use the light "Positron" style, which is drawn from OpenStreetMap data.
- The software used is Streamlit, pydeck/deck.gl, NumPy, SciPy (Qhull, csgraph Dijkstra, cKDTree), shapely, pandas, Altair and pytest.

12. Conclusion

Voronoi decides who serves whom, the KD-tree finds what is where, closest pair finds what is near, the convex hull shows how far the network reaches, and greedy set cover picks the sites worth keeping. Dijkstra on the real road network makes every distance honest, and Held-Karp with a 2-opt fallback turns a list of places that need a product into a route a rider can follow. Run on real OpenStreetMap data around Kalyani and drawn back on the street map, these algorithms turn a list of warehouse and customer coordinates into an interactive view of the delivery network, with each part solved by the Algorithms II technique that fits it.

The Hooghly shows why the road distance matters. With one road bridge along 27.5 km of river, a straight-line plan would send some west-bank customers to a warehouse whose real trip by van is more than three times as long. Measured along the roads, the service zones follow the river by themselves, and every one of those customers is served from its own bank.

Appendix A. Running the Project

The complete source code, data and tests are in the GitHub repository github.com/anipriyo/smart-delivery-network. The project uses Python 3.12.

The dashboard is also deployed on Streamlit Community Cloud at smart-delivery-network.streamlit.app, so it can be tried in a browser without installing anything. To run it locally, clone the repository and start it as follows.

```
git clone https://github.com/anipriyo/smart-delivery-network.git
cd smart-delivery-network
pip install -r requirements.txt
streamlit run app.py
```

The dashboard then opens in the browser on port 8501 of localhost. The OpenStreetMap extract, including the road graph, is part of the repository (`data/2026-10/`), so the algorithms need no download, and only the background street-map tiles need internet access.

Task	Command
Run the 59 tests	`python -m pytest tests/ -v`
Download the OSM extract again (current month)	`python scripts/fetch_osm.py`
Print every number used in this report	`python report/worked_example.py`
Retake the screenshots (with the app running)	`python report/take_screenshots.py`
Rebuild this PDF from `report/report.md`	`python report/build_pdf.py`

Repository layout

Path	Contents
`app.py` , `ui/`	The Streamlit dashboard, with its pages, deck.gl map layers, styling and the plain-words and pseudocode explanations (`ui/pseudo.py`)
`src/`	The algorithms (`convex_hull.py` , `closest_pair.py` , `kdtree.py` , `voronoi.py` , `set_cover.py`), the road network, zones by road and route planner (`roads.py`), the river model (`river.py`), geography and sampling (`geo.py` , `data.py`), and the Walkthrough traces (`traces.py`)
`tests/`	`test_algorithms.py` , `test_geo.py` , `test_river.py` , `test_roads.py`
`scripts/fetch_osm.py`	OpenStreetMap download and cleaning, including the road graph
`data/`	The October 2026 extract and hand-added ferries
`report/`	This report, its screenshots, the scripts that produce them, and the project overview document the work started from